ALGORITHM 1: Source-DPOR algorithm.

*// In all algorithms, we use ":=" notation for data shared among recursive calls and "***let***" for local variables.*

Initial call: $Explore(\langle\rangle, \emptyset)$

```
1  Explore(E, Sleep)
2      if ∃r ∈ (enabled(s_[E]) \ Sleep) then                    // If not at a maximal or "sleep set blocked" execution...
3          backtrack(E) := {r};                                  // ... initialize backtrackwith arbitrary r
4          while ∃p ∈ (backtrack(E) \ Sleep) do                  // Pick an unexplored p
5              foreach e ∈ dom(E) such that(e ≲_{E.p} next_[E](p)) do   // e and next_[E](p) race
6                  let E' = pre(E, e);
7                  let v = notdep(e, E).p;                       // Find events independent with e
8                  if I_[E'](v) ∩ backtrack(E') = ∅ then         // Ensure an initial exists in backtrack or...
9                      add some q' ∈ I_[E'](v) to backtrack(E');  // ... add an initial
10             let Sleep' = {q ∈ Sleep | E⊢p◇q};                 // Compute next sleep set
11             Explore(E.p, Sleep');                             // Recursively call Explore
12             add p to Sleep;                                   // Mark p as explored
```

execution sequences in which the race is reversed. Such sequences are obtained by leaving unchanged that part of E, which follows E' and depends neither on e nor on $next_{[E]}(p)$, and thereafter performing the event $next_{[E]}(p)$ instead of e. The unchanged part is obtained as $notdep(e, E)$. Note that some events in $notdep(e, E)$ may happen-before $next_{[E]}(p)$, and it is then necessary to perform them before $next_{[E]}(p)$ can be executed. To this, we add p in order to ensure that $next_{[E]}(p)$ is executed instead of e. Let v be $notdep(e, E).p$. By construction, any execution sequence that starts with $E'.v$ is guaranteed to be inequivalent to any execution sequence that starts with E. To ensure that an execution sequence of form $E'.v$ will be explored, the algorithm checks (at line 8) whether some process in $I_{[E']}(v)$ is already in $backtrack(E')$. If not, a process in $I_{[E']}(v)$ is added to $backtrack(E')$. This ensures that a sequence equivalent to $E'.v$ has been or will be explored.

The exploration (phase) is started recursively from $E.p$, using an appropriately initialized sleep set. According to rule (ii) in the explanation of sleep sets in Section 2, the sleep set for the exploration of $E.p$ should be initialized to be the set of processes currently in the sleep set of E that are independent with p after E (i.e., $Sleep' = \{q \in Sleep \mid E \vdash q \diamond p\}$).

On Source Sets and Persistent Sets. The mechanism by which source-DPOR produces source sets rather than persistent sets is the test at line 8. In DPOR algorithms based on persistent sets, such as those of Flanagan and Godefroid [2005b], Lauterburg et al. [2010], Tasharofi et al. [2012], Sen and Agha [2006], and Saarikivi et al. [2012], this test must be stronger, and at least guarantee that $backtrack(E')$ contains a process q such that q performs some event in v which "happens-before" $next_{[E]}(p)$ in $E.p$. Thus, if the test at line 8 and the addition at line 9 are strengthened into

$$\begin{array}{l}\textbf{if } \nexists e' \in dom_{[E']}(v).\ \left[e' \rightarrow_{E.p} next_{[E]}(p) \wedge \widehat{e'} \in (I_{[E']}(v) \cap backtrack(E'))\right] \textbf{ then} \\ \quad \text{add } \widehat{e'} \text{ for some } e' \in dom_{[E']}(v) \text{ with } \left[e' \rightarrow_{E.p} next_{[E]}(p) \wedge \widehat{e'} \in I_{[E']}(v)\right] \text{ to } backtrack(E')\end{array}$$

then the algorithm ensures that $backtrack(E')$ will be a persistent set when $Explore(E, Sleep)$ returns. These lines guarantee that the first event in v which is dependent with some process in $backtrack(E')$ is performed by some process in $backtrack(E')$, thus making $backtrack(E')$ a persistent set. In contrast, our test at line 8 does not require the added process to perform an event which "happens-before" $next_{[E]}(p)$. Consider, for instance, that v is just the sequence $q.p$, where q is independent